Universidad Nacional de Córdoba
Facultad de Ciencias Exactas, Físicas y Naturales
Ingeniería en Computación

Redes de Computadoras

Monografía

MQTT y Protocol Buffers

Un stack de comunicación en tiempo real para IoT, robótica y mensajería con servicios en la nube

Equipo
WireGuardians

Viberti, Benjamin[1]	DNI 46224179
Espinoza Sutta, Aaron Alejandro[1]	DNI 96009173
Cleri, Juan Ignacio[1]	DNI 46452662
Pineda, Juan Ignacio[1]	DNI 45591343
Grafión, Atilio Leonel[1]	DNI 43940195
Badenes, Tomás[1]	DNI 44785038
Oviedo, Ignacio Nicolás[1]	DNI 44169239
Mendez, Jorge Nicolas[1]	DNI 41301342

Docente
Oliva Cuneo, Facundo Nicolas

[1]{b.viberti,aaron.espinoza_4500,ignacio.cleri,juan.ignacio.pineda,atilio.grafion,tomasbadenes,
ignacio.oviedo.239,jorge.mendez}@mi.unc.edu.ar

BORRADOR

26 de octubre de 2026

Índice

Introducción **3**
1. Objetivos 4
2. Alcance 4
Marco teórico **5**
1. Modelo publicador/suscriptor 5
2. MQTT 5
3. Protocol Buffers 5
4. Stack MQTT y Protocol Buffers 5
5. Comparación con alternativas 5
   5.1. MQTT, HTTP y CoAP 5
   5.2. Protocol Buffers y JSON 5
Desarrollo **6**
1. MQTT sobre TCP y TLS 6
   1.1. Conexión persistente y keep-alive 6
   1.2. Cifrado con TLS 6
   1.3. QoS MQTT sobre la confiabilidad de TCP 6
2. Capa de red 6
   2.1. NAT y firewalls 6
   2.2. Ruteo interno 6
   2.3. Ruteo externo 6
   2.4. Convergencia y conexiones de larga duración 6
3. Ruteo por tópicos y ruteo IP 6
4. Casos de uso 6
5. Diseño experimental 6
   5.1. Plataforma común 6
   5.2. Experimento 1: MQTT contra HTTPS para telemetría 6
   5.3. Experimento 2: sincronización de osciladores acoplados 6
Conclusiones **10**
Bibliografía **11**
Anexos **12**
A. Mensajes del experimento de sincronización 12

BORRADOR

Introducción

Gran parte de la Internet comercial se apoya en el modelo de solicitud y respuesta de HTTP: un cliente consulta a un servidor cuando necesita un dato y recibe la respuesta, que suele venir codificada en formatos de texto como HTML, JSON o XML. Este modelo está pensado para intercambios puntuales, que el cliente inicia a demanda, y no para flujos de datos continuos ni para que el servidor tome la iniciativa.

Un número creciente de sistemas no encaja en ese modelo. Los dispositivos del Internet de las Cosas, las flotas de robots y las aplicaciones de mensajería móvil generan flujos continuos de telemetría, eventos y comandos a alta frecuencia y en ambos sentidos, desde equipos de recursos acotados que operan sobre redes celulares o inalámbricas inestables y detrás de firewalls fuera del control de quien administra el servidor. Esta combinación plantea dos limitaciones de fondo para la ingeniería de redes: el costo, en ancho de banda y energía, de consultar periódicamente y de codificar los datos en formatos de texto; y la imposibilidad de que un servidor inicie comunicaciones hacia dispositivos que no admiten conexiones entrantes.

Frente a este panorama, muchos de estos sistemas han recurrido a la mensajería de tipo publicador/suscriptor, que desacopla a los participantes mediante un intermediario. En este campo se inscribe la combinación de MQTT y Protocol Buffers, un stack pensado para la comunicación en tiempo real con ancho de banda limitado. MQTT es un protocolo liviano de transporte de mensajes, organizado en torno a un broker, concebido para entornos restringidos (como los de comunicación máquina a máquina y de Internet de las Cosas) en los que se necesita una huella de código reducida o el ancho de banda es escaso. Protocol Buffers es un mecanismo, independiente del lenguaje y de la plataforma, para serializar datos estructurados y tipados en un formato más compacto y rápido que los textuales, a partir de un esquema compartido.

Su adopción en sistemas reales es amplia y heterogénea. En la mensajería a gran escala, Facebook recurrió a MQTT en Messenger para sostener una conexión persistente con sus servidores sin agotar la batería de los teléfonos, lo que permitió, en muchos casos, entregar los mensajes en cientos de milisegundos en lugar de varios segundos. En robótica, InOrbit, una empresa que ofrece una plataforma de orquestación de flotas de robots, implementa la comunicación entre servicios e integraciones desplegadas *on the edge* utilizando MQTT y mensajes definidos con Protocol Buffers, donde brokers desplegados en la nube reciben datos en tiempo real de miles de robots y dispositivos IoT, que a su vez son controlados por el mismo canal, aun en instalaciones con redes inestables. Desarrollado por la misma empresa, el producto de código abierto OpenRobOps libera las definiciones de su protocolo de comunicación en estructuras Protobuf que fijan el formato de las comunicaciones entre robot y plataforma.

Un rasgo de este esquema es especialmente relevante desde el encaminamiento: el dispositivo abre una conexión persistente hacia el broker alojado en la nube, cuya dirección suele estar habilitada en el firewall, y por esa misma conexión el broker le entrega los mensajes, sin necesidad de conexiones entrantes. El principio rector del stack es, entonces, trasladar la complejidad desde la red hacia el protocolo de aplicación: ningún participante necesita conocer la ubicación de los demás, sino solo la del broker, y el contrato sobre el formato de los datos se fija una única vez en un esquema distribuido entre los clientes. De este modo, la comunicación en tiempo real entre miles de entidades heterogéneas resulta viable sobre redes que no fueron concebidas para ese fin.

BORRADOR

1. Objetivos

Se desea realizar un análisis riguroso del stack de comunicación formado por MQTT y Protocol Buffers, como alternativa para el intercambio de datos en tiempo real, a alta frecuencia y con ancho de banda limitado, en aplicaciones de Internet de las Cosas, robótica y mensajería. El estudio sigue el recorrido de un mensaje desde el dispositivo que lo genera hasta el broker que lo distribuye, y evalúa cómo cada capa de la arquitectura contribuye a que ese recorrido sea eficiente y confiable.

Más que catalogar las prestaciones de ambas tecnologías, el trabajo busca explicar por qué resultan adecuadas para el problema planteado: sostener una comunicación continua sobre redes que el diseñador no controla, con enlaces inestables, firewalls de terceros y trayectos que atraviesan múltiples sistemas autónomos. Con ese fin, el análisis se articula en torno a dos ejes centrales de la materia, que pertenecen a capas distintas: la capa de aplicación (capa 7), donde operan MQTT y Protocol Buffers, y la capa de red (capa 3), donde el ruteo determina por qué camino llegan los mensajes al broker y cómo se restablece ese camino cuando la red cambia.

Para alcanzar este propósito se plantean los siguientes objetivos específicos:

- Presentar el modelo publicador/suscriptor y describir cómo lo implementa MQTT (broker, tópicos, niveles de calidad de servicio y sesiones), junto con el mecanismo de serialización de Protocol Buffers, y compararlos con alternativas como HTTP, CoAP y JSON.
- Analizar el funcionamiento de MQTT sobre la capa de transporte (TCP, keep-alive y TLS) y distinguir las garantías de entrega de sus niveles de calidad de servicio de la confiabilidad que ya ofrece TCP.
- Analizar el recorrido de los mensajes en la capa de red (capa 3): NAT y firewalls, y el efecto de la convergencia de los protocolos de ruteo interno (OSPF, con RIP y EIGRP como comparación) y externo (BGP) sobre las conexiones TCP de larga duración en las que se apoya MQTT.
- Contrastar el ruteo por destino IP de la capa de red con el ruteo por tópicos que realiza el broker en la capa de aplicación.
- Examinar casos de uso reales en robótica, mensajería e Internet de las Cosas, como los de InOrbit, para determinar las ventajas y limitaciones del stack en entornos de alta frecuencia y recursos acotados.
- Medir experimentalmente, sobre nodos emulados, el costo en bytes y latencia de MQTT y Protocol Buffers frente a HTTPS y JSON, y su comportamiento ante pérdidas, latencia y cortes de conexión.

2. Alcance

BORRADOR

Marco teórico

1. Modelo publicador/suscriptor

2. MQTT

3. Protocol Buffers

4. Stack MQTT y Protocol Buffers

5. Comparación con alternativas

5.1. MQTT, HTTP y CoAP

5.2. Protocol Buffers y JSON

BORRADOR

Desarrollo

1. MQTT sobre TCP y TLS

1.1. Conexión persistente y keep-alive

1.2. Cifrado con TLS

1.3. QoS MQTT sobre la confiabilidad de TCP

2. Capa de red

2.1. NAT y firewalls

2.2. Ruteo interno

2.3. Ruteo externo

2.4. Convergencia y conexiones de larga duración

3. Ruteo por tópicos y ruteo IP

4. Casos de uso

5. Diseño experimental

5.1. Plataforma común

5.2. Experimento 1: MQTT contra HTTPS para telemetría

5.2.1. Planteo

5.2.2. Implementación

5.2.3. Resultados

5.2.4. Análisis de resultados

5.3. Experimento 2: sincronización de osciladores acoplados

5.3.1. Planteo

El modelo. Cada oscilador tiene una frecuencia natural ω_i, que es fija, y una fase θ_i, que evoluciona según

$$\dot{\theta}_i = \omega_i + \frac{K}{N}\sum_{j=1}^{N}\sin\big(\theta_j - \theta_i\big), \quad i = 1, ..., N \tag{1}$$

donde K es la intensidad del acoplamiento, en rad/s. El acoplamiento cambia la velocidad instantánea $\dot{\theta}_i$: si el oscilador j va adelantado, $\sin\big(\theta_j - \theta_i\big) > 0$ y el oscilador i se acelera; si va atrasado, se frena.

Parámetro de orden y campo medio. El grado de sincronía se mide con el centroide de las fases sobre el círculo unitario:

BORRADOR

$$re^{i\psi} = \frac{1}{N}\sum_{j=1}^{N} e^{i\theta_j} \tag{2}$$

r vale 0 si las fases están dispersas y 1 si coinciden; ψ es la fase media. Con esta definición, la Ecuación 1 se reescribe en forma de campo medio, que vale para cualquier N:

$$\dot{\theta}_i = \omega_i + Kr\sin(\psi - \theta_i) \tag{3}$$

Cada oscilador es atraído hacia la fase media con una fuerza Kr. Es la forma que se implementa en los nodos. El promedio de fases tiene que ser circular: se suman los vectores $(\cos\theta_j, \sin\theta_j)$ y se toman el módulo y el ángulo de la suma. Promediar los ángulos directamente falla en el corte entre 2π y 0.

Frecuencia común. Como el acoplamiento es simétrico, al sumar todas las ecuaciones los senos se cancelan de a pares:

$$\frac{d}{dt}\sum_i \theta_i = \sum_i \omega_i \quad \Rightarrow \quad \Omega = \overline{\omega} = \frac{1}{N}\sum_i \omega_i \tag{4}$$

Si el grupo se engancha, gira a la frecuencia media Ω. Desde ahí se trabaja con las desintonías $\Delta_i = \omega_i - \overline{\omega}$. Con retardo en el acoplamiento, esta propiedad deja de valer.

Límite de infinitos osciladores. Con frecuencias tomadas de una distribución $g(\omega)$ unimodal y simétrica, aparece una transición en el acoplamiento crítico

$$K_c = \frac{2}{\pi g(0)} \tag{5}$$

Para una distribución de Lorentz de ancho γ, $K_c = 2\gamma$ y, por encima del umbral, $r = \sqrt{1 - K_c/K}$. Con pocos nodos este resultado queda solo como contexto, y K_c se calcula como se muestra a continuación.

Umbral exacto con N finito. En el marco que gira a $\overline{\omega}$, un estado enganchado cumple

$$\Delta_i = u\sin(\theta_i - \psi), \quad u = Kr \tag{6}$$

Cada oscilador necesita $|\Delta_i| \le u$ y queda con un desfasaje $\theta_i - \psi = \arcsin(\Delta_i/u)$. Reemplazando en la Ecuación 2, con todos los cosenos positivos, se obtiene la condición de autoconsistencia:

$$r = \frac{u}{K} = f(u) = \frac{1}{N}\sum_{i=1}^{N}\sqrt{1 - \left(\frac{\Delta_i}{u}\right)^2}, \quad u \ge \max_i|\Delta_i| \tag{7}$$

El estado enganchado existe si K alcanza el mínimo de $u/f(u)$:

$$K_c = \min_{u \ge \max_i|\Delta_i|} \frac{u}{f(u)} \tag{8}$$

Con esto se calculan, para las frecuencias concretas de cada corrida, el umbral de enganche total, el r final y el desfasaje de cada nodo. Con $N = 2$ se obtiene $K_c = |\Delta\omega|$.

$N = 3$ simétrico. Con frecuencias $\overline{\omega} - \Delta$, $\overline{\omega}$ y $\overline{\omega} + \Delta$, por simetría el oscilador central queda alineado con ψ y los otros dos a $\pm\alpha$. La condición de equilibrio del oscilador rápido es

$$\Delta = \frac{K}{3}(\sin\alpha + \sin 2\alpha) \tag{9}$$

BORRADOR

El máximo de $\sin\alpha + \sin 2\alpha$ es aproximadamente 1,760, en $\alpha \approx 53{,}6°$, de modo que $K_c \approx 1{,}704\ \Delta$. El oscilador central hace de puente entre los extremos.

Enganche parcial y normalización. Con $N \geq 3$ y frecuencias asimétricas, puede engancharse un subgrupo mientras otro nodo deriva. Este caso no tiene fórmula simple y se estudia por simulación, comparando las frecuencias efectivas de cada nodo. Como el acoplamiento se divide por N, al agregar nodos con el mismo K cada interacción individual se debilita. Si cada nodo dividiera por la cantidad de fases recibidas en lugar de un N fijo, el modelo cambiaría cuando se pierden mensajes; por eso conviene usar N fijo.

Efecto de la red: retardo. En la red, cada nodo conoce las fases de los demás con un retardo τ:

$$\dot{\theta}_i = \omega_i + \frac{K}{N} \sum_{j \neq i} \sin\big(\theta_j(t-\tau) - \theta_i(t)\big) \tag{10}$$

La suma excluye $j = i$ porque cada nodo conoce su propia fase sin demora. Para osciladores idénticos, si se incluyera ese término propio, la frecuencia común cumpliría $\Omega = \omega - K\sin(\Omega\tau)$, y el estado en fase sería estable si $K\cos(\Omega\tau) > 0$. En la implementación ese término no existe.

$N = 2$ con retardo. Con $\varphi = \theta_2 - \theta_1$ y $\Delta\omega = \omega_2 - \omega_1$, el estado enganchado de la Ecuación 10 cumple

$$\Delta\omega = K\sin\varphi\cos(\Omega\tau), \quad \Omega = \overline{\omega} - \frac{K}{2}\cos\varphi\sin(\Omega\tau) \tag{11}$$

La primera condición solo tiene solución si $K\,|\cos(\Omega\tau)| \geq |\Delta\omega|$. Aproximando $\Omega \approx \overline{\omega}$:

$$K_c(\tau) \approx \frac{|\Delta\omega|}{|\cos(\overline{\omega}\tau)|} \tag{12}$$

Una simulación con $\overline{\omega} = 2\pi$ rad/s y $\Delta\omega = 0{,}2$ rad/s dio el primer K con enganche en 0,200 sin retardo, 0,247 con $\tau = 100$ ms y 0,647 con $\tau = 200$ ms, y la frecuencia común bajó de 6,283 a 6,178 rad/s. La Ecuación 12 da así una predicción de K_c y de Ω en función de la latencia.

El retardo entra multiplicado por la frecuencia absoluta de parpadeo $\overline{\omega}$, no por la desintonía. A 1 Hz, con 250 ms de latencia ya no hay enganche con un K moderado; a 0,5 Hz el límite pasa a 500 ms. La frecuencia de parpadeo define cuánta latencia tolera el sistema.

Pérdida de paquetes. MQTT viaja sobre TCP: si se descarta un paquete, TCP lo retransmite y el mensaje llega igual, más tarde, aun con QoS 0. La pérdida de paquetes se traduce en retardo adicional y jitter, no en mensajes perdidos. Los mensajes se pierden de verdad solo si se descartan en la aplicación o si se corta la conexión. En ese caso, el efecto depende de la política del nodo:

- si mantiene la última fase recibida, la pérdida aumenta la edad de la información y actúa como retardo adicional;
- si usa solo lo recibido en cada paso, con N fijo, el acoplamiento efectivo baja a aproximadamente $K(1-p)$.

Actualización discreta. Cada nodo integra su fase con un paso propio y publica con un período h. Son dos parámetros distintos. Con integración de Euler, el lazo es estable si $0 < Kh < 2$, pero con valores reales $Kh \approx 0{,}05$, así que ese límite no influye. El período de publicación determina cada cuánto llega información nueva, y actúa como un retardo adicional de alrededor de $h/2$.

BORRADOR

Jitter y caída del broker. La variación del retardo actúa como ruido en la fase: baja r en régimen y sube el K_c medido. Si cae el broker, el acoplamiento se anula y cada nodo vuelve a su frecuencia natural.

Acoplamiento efectivo. K es una ganancia del software de cada nodo: se fija al comienzo de cada corrida y no cambia durante ella. No depende de la red y no cambia el tráfico, que lo determinan h y el tamaño de los mensajes. La red no modifica K, pero sí el acoplamiento efectivo (Tabla 1). Por eso hace falta un K mayor que el teórico para lograr el enganche, y la diferencia entre el K_c teórico y el medido es lo que el stack le quita a la sincronización.

Tabla 1: Efecto de la red sobre el acoplamiento efectivo.

Efecto de la red	**Acoplamiento efectivo**
Latencia τ	Con $N = 2$, el enganche exige $K \, \lvert\cos(\overline{\omega}\tau)\rvert \geq \lvert\Delta\omega\rvert$
Pérdida de mensajes (solo lo recibido, N fijo)	Aproximadamente $K(1-p)$
Período de publicación h	Información vieja: retardo adicional de alrededor de $h/2$
Caída del broker	Nulo

5.3.2. Implementación

5.3.3. Resultados

5.3.4. Análisis de resultados

BORRADOR

Conclusiones

BORRADOR

Bibliografía

[1] A. Banks, E. Briggs, K. Borgendale, y R. Gupta, Eds., «MQTT version 5.0», OASIS, OASIS Standard, 2019. [En línea]. Disponible en: https://docs.oasis-open.org/mqtt/mqtt/v5.0/mqtt-v5.0.html

[2] P. T. Eugster, P. A. Felber, R. Guerraoui, y A.-M. Kermarrec, «The many faces of publish/subscribe», *ACM Computing Surveys*, vol. 35, n.º 2, pp. 114-131, 2003, doi: 10.1145/857076.857078.

[3] Google, «Overview». Accedido: 19 de octubre de 2026. [En línea]. Disponible en: https://protobuf.dev/overview/

[4] L. Zhang, «Building Facebook Messenger». [En línea]. Disponible en: https://engineering.fb.com/2011/08/12/android/building-facebook-messenger/

[5] InOrbit, «InOrbit.AI: We make robots work». Accedido: 19 de octubre de 2026. [En línea]. Disponible en: https://www.inorbit.ai/

[6] inorbit-ai, «edge-sdk-python: InOrbit Python Edge SDK». Accedido: 19 de octubre de 2026. [En línea]. Disponible en: https://github.com/inorbit-ai/edge-sdk-python

[7] OpenRobOps, «OpenRobOps: Open source robot fleet management». Accedido: 19 de octubre de 2026. [En línea]. Disponible en: https://openrobops.org/

[8] OpenRobOps/oro, «oro: OpenRobOps open-source robot fleet management and operations platform». Accedido: 19 de octubre de 2026. [En línea]. Disponible en: https://github.com/OpenRobOps/oro

BORRADOR

Anexos

A. Mensajes del experimento de sincronización

BORRADOR